

# Trajectory Planning for Mission Completion Time Reduction in UAV-Assisted Data Collection in Wireless Sensor Networks

Bo-Syun Ke

Department of Electrical Engineering

National Cheng Kung University

Unmanned aerial vehicles (UAVs) provide a flexible solution for data collection in wireless sensor networks (WSNs), especially when ground nodes (GNs) are deployed in wide or hard-to-access areas. Mission completion time (MCT) is an important performance metric in UAV-assisted data collection tasks. Existing studies have mainly focused on UAV flight trajectory planning to reduce mission completion time. But the data collection process also has a significant impact on MCT because the transmission rate varies with the distance between the UAV and GNs.

This thesis proposes a trajectory planning method for a single-UAV-assisted WSN data collection task. The proposed method first obtains a shortest geometric reference path that passes through the GN communication regions and uses this path as global directional guidance for local trajectory planning. For each GN, the UAV adjusts its data collection trajectory according to the required data amount. For adjacent GNs with overlapping communication regions, the proposed method further allows service switching within the overlap region to enable a more efficient transition to the next GN. Simulation results show that the proposed method outperforms the representative benchmark methods.

**Keywords**—UAV, mission completion time, wireless sensor networks, trajectory planning, data collection.

# Acknowledgments

First and foremost, I would like to express my deepest gratitude to my advisor, Professor Kuo-Feng Ssu, for his guidance, encouragement, and support throughout my academic journey at National Cheng Kung University. His invaluable advice and continuous support greatly contributed to the completion of this thesis. I would also like to thank Professors Hewijin Christine Jiau and Chia-Hung Kao for their valuable suggestions and for serving as members of my thesis committee. Their comments and feedback have helped improve the quality of this research. My sincere thanks also go to the members of the Dependable Computing Laboratory, including Yu-Jung Chang, Su Wu, You-Yao Chen, Hao-Chen Liu, Yen-Wei Lin, Xing-Ya Yang, Yu-Wei Zheng, Yun-Gung Chung, Zih-Yun Lin, Tzu-Hau Chen, and Yin-Wei Chiu, for their helpful advice, discussions, and support during my research. Finally, I am deeply grateful to my family and friends for their constant support, companionship, and encouragement. Their understanding and encouragement have given me the strength to complete this thesis.

# Table of Contents

## Chapter

|          |                                                               |           |
|----------|---------------------------------------------------------------|-----------|
| <b>1</b> | <b>Introduction</b>                                           | <b>1</b>  |
| <b>2</b> | <b>Related Work</b>                                           | <b>3</b>  |
| <b>3</b> | <b>System Model</b>                                           | <b>6</b>  |
| 3.1      | System Overview                                               | 6         |
| 3.2      | System Assumption                                             | 7         |
| 3.3      | Problem Formulation                                           | 9         |
| <b>4</b> | <b>Proposed Solution</b>                                      | <b>13</b> |
| 4.1      | Notation and Definitions                                      | 13        |
| 4.2      | Routing Table Establishment                                   | 13        |
| 4.3      | Phase I: Routing Analysis with Concurrent Payment and Probing | 16        |
| 4.3.1    | Path Scoring                                                  | 16        |
| 4.3.2    | Channel Usage Frequency Update Mechanism                      | 17        |
| 4.3.3    | Trusted Nodes for Real-Time Channel State Inquiry             | 18        |
| 4.3.4    | Adaptive Multi-Path Payment Splitting Mechanism               | 19        |
| 4.3.5    | Path Probing and Payment Execution Procedure                  | 20        |
| 4.4      | Phase II: Failed Handling                                     | 20        |
| <b>5</b> | <b>Performance Evaluation</b>                                 | <b>23</b> |
| 5.1      | Comparison Algorithms                                         | 24        |
| 5.2      | Performance Under Different Workloads                         | 24        |
| 5.3      | Performance with Scaled Capacities                            | 27        |
| 5.4      | Performance with Failed Handling Mechanisms                   | 30        |
| <b>6</b> | <b>Conclusion and Future Work</b>                             | <b>32</b> |
| 6.1      | Conclusion                                                    | 32        |
| 6.2      | Future Work                                                   | 33        |
|          | <b>References</b>                                             | <b>34</b> |

# List of Figures

|     |                                                                        |    |
|-----|------------------------------------------------------------------------|----|
| 3.1 | System overview. . . . .                                               | 7  |
| 3.2 | Example of the payment channel. . . . .                                | 9  |
| 3.3 | Payment channel network. . . . .                                       | 10 |
| 4.1 | Flow chart of the system. . . . .                                      | 14 |
| 5.1 | Success rate under different workloads. . . . .                        | 25 |
| 5.2 | Execution time under different workloads. . . . .                      | 25 |
| 5.3 | Transaction fees under different workloads. . . . .                    | 26 |
| 5.4 | Success rate under different channel sizes. . . . .                    | 28 |
| 5.5 | Execution time under different channel sizes. . . . .                  | 28 |
| 5.6 | Transaction fees under different channel sizes. . . . .                | 29 |
| 5.7 | Success rate with and without failed handling mechanisms. . . . .      | 30 |
| 5.8 | Successful volume with and without failed handling mechanisms. . . . . | 31 |

# List of Tables

|     |                                 |    |
|-----|---------------------------------|----|
| 4.1 | List of Notations . . . . .     | 14 |
| 5.1 | Simulation Parameters . . . . . | 24 |

# Chapter 1

## Introduction

With the rapid development of the Internet of Things (IoT) and wireless communication technologies, wireless sensor networks (WSNs) have been widely applied in environmental monitoring, smart agriculture, post-disaster detection, military reconnaissance, and other fields [1], [2], [3]. In these applications, ground nodes (GNs) are mainly responsible for collecting environmental parameters, such as temperature, humidity, air quality, and water quality. In such IoT applications, each GN usually generates lightweight data packets [14], [15], [16]. However, due to limited battery capacity and hardware cost, GNs are often unable to support long-distance wireless transmission. Therefore, transmitting sensing data back to a data center remains a major challenge [3].

1. A trajectory planning method is developed to reduce the MCT of single-UAV-assisted WSN data collection by considering both the inter-region flight path and the intra-region data collection trajectory.
2. A global-direction-guided local trajectory adjustment method is introduced. The method enables the UAV to adjust the data collection trajectory within each GN

while remaining close to the shortest geometric path, thereby reducing unnecessary flight distance.

3. A flexible service switching strategy is developed for consecutive GNs with overlapping communication ranges. Instead of restricting service switching to the boundary of an individual communication circle, the proposed strategy allows the UAV to utilize the shared coverage region between neighboring GNs as a switching area, thereby reducing redundant boundary transitions and unnecessary back-and-forth movement.
4. Simulation results show that the proposed method reduces the overall MCT compared with the benchmark methods under different data requirements and GN distributions. The results further indicate that global geometric guidance and overlap-aware service switching contribute to reducing both flight distance and mission completion time.

The remainder of this thesis is organized as follows. Chapter 2 reviews related studies on trajectory planning for UAV-assisted WSN data collection. Chapter 3 introduces the system model, including the communication model and the UAV flight model. Chapter 4 describes the proposed trajectory planning method. Chapter 5 presents the simulation setup, benchmark methods, and performance evaluation. Finally, Chapter 6 concludes this thesis and discusses possible future research directions.

## Chapter 2

# Related Work

Routing within PCNs presents a significant research challenge, primarily due to limited channel capacity and highly dynamic network conditions. Most current routing algorithms necessitate that the initiating node possesses global network topology information to compute end-to-end payment paths from sender to receiver. These methods typically rely on path probing to assess residual channel capacities before transaction execution.

For instance, Deter-Pay [?] employs a greedy strategy and probes to select feasible paths with the lowest fees. Similarly, CoinExpress [?] and Flash [?] also leverage probing-based approaches. Flash segments large payments into multiple sub-payments and probes channel states to acquire necessary information prior to execution. EPA-Route [?] further proposes a routing and probing mechanism that simultaneously balances high success rates with low transaction fees. EPA-Route uses local routing tables at each node to identify potential paths. It then filters and prunes these paths based on hop feasibility. After path discovery, EPA-Route aggregates multiple valid paths. It segments payments into sub-payments and distributes them across these paths to

enhance overall efficiency. However, this “probe-then-pay” approach inherently introduces substantial transaction delays, as routing decisions are contingent upon the completion of the probing process.

Beyond probe-based methods, several studies [?, ?, ?] concentrate on enhancing routing efficiency and channel longevity through dynamic fee adjustments. The core idea is to increase fees for unbalanced or heavily utilized channels while lowering fees for balanced channels. This guides traffic away from potential bottleneck paths. Fence [?] exemplifies this approach by setting strategic fee rates. This prevents rapid fund depletion on one side of a channel due to asymmetric payment flows. PCN sustainability and long-term throughput are enhanced. MPCN-RP [?] further incorporates diverse transaction costs. The system integrates channel imbalance and hop fees into its routing model to identify paths optimized for both total cost and path length. Despite the mechanisms for guiding traffic and balancing funds, the implementation encounters challenges. Frequent fee fluctuations introduce uncertainty in transaction costs. This is particularly detrimental to high-frequency or time-sensitive applications. Additionally, malicious nodes could potentially manipulate fees to disrupt normal routing, and then security risks are introduced.

Current literature primarily concentrates on enhancing initial payment success rates. Methods, such as AutoTune [?], PACO [?], and VEIN [?], aim to enhance path selection efficiency and resource utilization. Despite advancements in routing design, the systematic exploration of retransmission strategies following payment failures remains limited. This limitation is particularly critical given the constraints of limited channel capacity.

Existing research has made notable strides in enhancing initial transaction success rates and fee adjustment strategies. The reliance on real-time channel state probing introduces substantial delays. Furthermore, the recovery and retransmission strategies for payment failures remain a critical issue. This study aims to develop low-latency routing mechanisms and robust retransmission protocols for enhancing the scalability and reliability of PCNs.

## Chapter 3

# System Model

This chapter introduces the system model of the PCN, including its structure, routing mechanisms, and related assumptions.

### 3.1 System Overview

The proposed system aims to significantly enhance the overall transaction success rate, reduce transaction fees, and minimize transaction latency. Transaction latency is specifically defined as the duration from the initiation of a transaction request until either the complete delivery of funds to the recipient or the system’s confirmation of failure.

Upon receiving a transaction request, the system employs a routing algorithm to identify optimal payment pathways. A transaction is considered successful only when the entire requested amount reaches the recipient. This algorithm is designed to increase the success rate while carefully balancing transaction cost and latency, ensuring transactions complete with reduced fees without compromising reliability or incurring substantial delays.

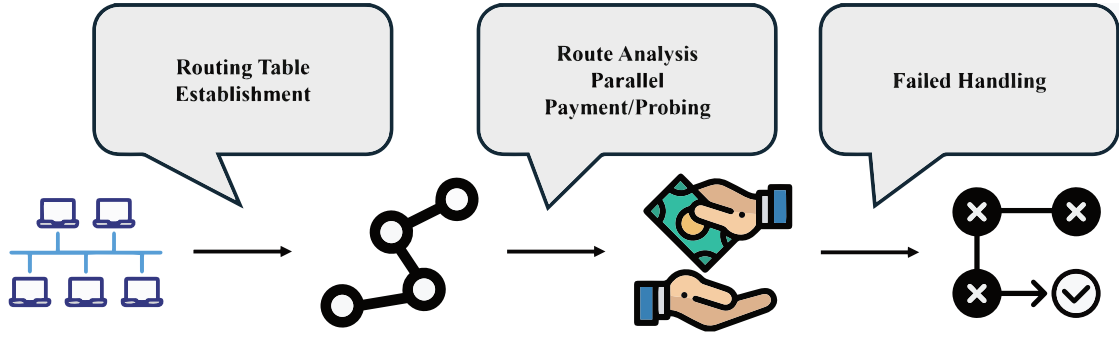


Figure 3.1: System overview.

Figure 3.1 depicts an overview of the system. The routing algorithm comprises three primary components: Routing Table Establishment, Route Analysis with Parallel Payment Probing, and Failure Handling. Each component will be elaborated upon in subsequent sections.

## 3.2 System Assumption

The PCN serves as a Layer-2 solution designed to enhance the scalability of blockchain transactions and reduce latency. The core concept involves establishing payment channels that enable two parties to perform multiple off-chain transactions without submitting each transaction to the underlying blockchain. Individual channels are interconnected to form a network. This network enables efficient value transfer across multiple nodes. To incentivize nodes to participate in routing services, most protocols implement a routing fee mechanism, which allows intermediary nodes to earn a reward for forwarding transactions. This section will introduce the concepts of payment channels, the structure of the PCN, and the design of routing fees in subsequent subsections.

**Payment Channels:** A payment channel is jointly established by two participants through the deployment of a smart contract. Both parties deposit funds into a 2-of-2 multi-signature wallet that serves as the basis of the channel. The opening and closing

of the channel are recorded on the underlying blockchain, and all transactions during the channel's lifetime are conducted off-chain. This multi-signature wallet facilitates a bidirectional payment channel. Each participant holds a certain amount of available balance. In each transaction, one party's balance decreases while the other party's balance increases by the corresponding amount.

As depicted in Figure 3.2a, Alice and Bob deposit funds into the payment channel. Subsequently, Alice sends 1 satoshi to Bob, as illustrated in Figure 3.2b. Following this transaction, as shown in Figure 3.2c, Alice's balance decreases by 1 satoshi, and Bob's balance increases by 1 satoshi, so completing an off-chain transfer of value is completed.

**Payment Channel Network:** A payment channel network comprises multiple payment channels and participating users. In instances where no direct payment channel exists between a sender and a receiver, funds can be routed through intermediate nodes. As depicted in Figure 3.3, nodes within the network can forward funds to other nodes via connected intermediate nodes and their corresponding payment channels.

This study assumes that the network topology of the PCN is periodically disseminated and updated among nodes using a gossip protocol. Each node can access information, such as the overall network topology, the fee models of other nodes, and the total capacity of payment channels. However, the real-time channel balances are considered private and cannot be directly obtained.

**Transaction Fee:** In a PCN, the transaction fee typically comprises two components: a *base fee* and a *proportional fee*. The total transaction fee model can be expressed as follows.

Let each path  $q$  be defined as a sequence of adjacent payment channels  $(i, j)$ . For each channel  $(i, j)$ , the following are defined:

- $\beta_{i,j}$ : the *base fee* charged by channel  $(i, j)$ ;
- $\gamma_{i,j}$ : the *proportional fee rate* set by channel  $(i, j)$ .

Then, the total fee incurred by sending an amount  $z_{\rho,q}$  along path  $q$  is given by:

$$C(z_{\rho,q}, q) = \sum_{(i,j) \in q} (\beta_{i,j} + \gamma_{i,j} \cdot z_{\rho,q}) \quad (1)$$

This model encapsulates the fact that the total transaction fee for a sub-transaction is contingent upon the number of hops along the selected path, the fee configuration of each channel, and the amount being transferred. The model is widely adopted in Layer-2 payment protocols, including the Lightning Network.

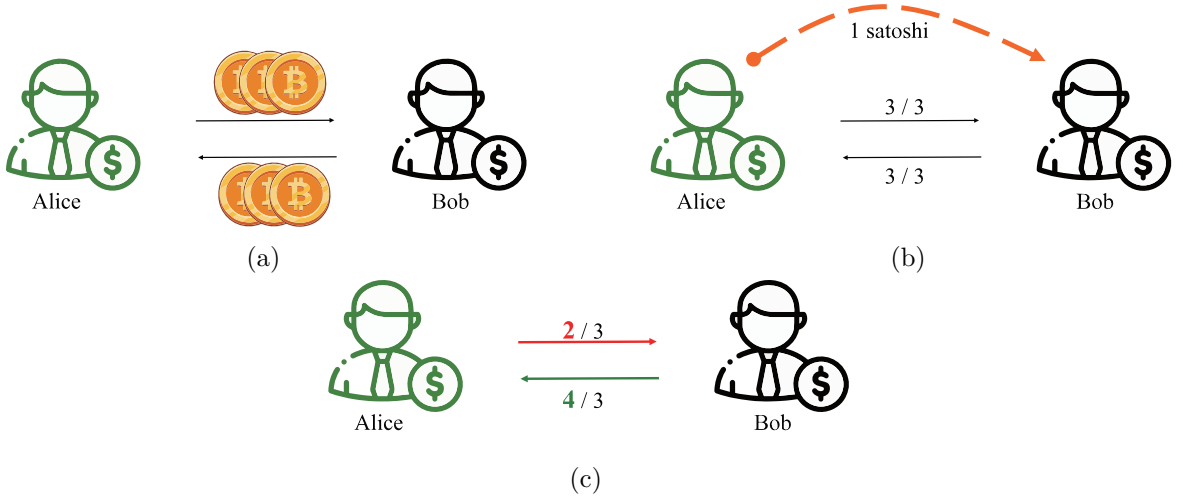


Figure 3.2: Example of the payment channel.

### 3.3 Problem Formulation

This study investigates a PCN environment where multiple transactions execute concurrently within a fixed time interval  $T$ . Each node within the network may process

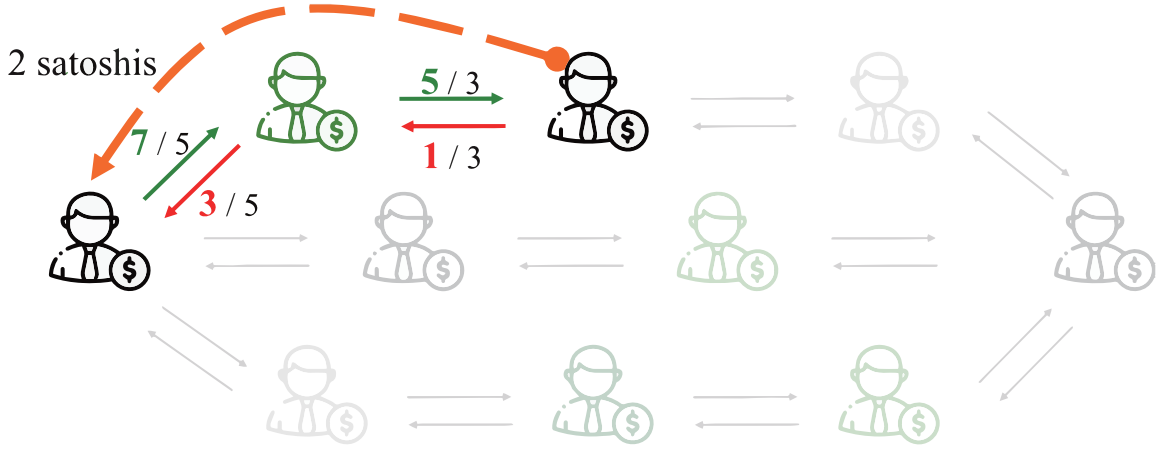


Figure 3.3: Payment channel network.

multiple incoming fund transfer requests in parallel. A key operational characteristic is that intermediate channels involved in a transaction remain locked until the fund transfer is finalized. Given the dynamic and unpredictable nature of individual channel balances, it is challenging to ascertain whether a path possesses sufficient liquidity using only predefined or static information. To address this challenge, a routing algorithm is proposed to optimize path selection and transaction amount allocation within such a dynamic environment.

Let  $\mathcal{R}$  denote the set of all transaction requests generated during the interval  $T$ . Each transaction request  $\rho \in \mathcal{R}$  is represented as a tuple  $\rho = (u_\rho, v_\rho, a_\rho, t_\rho)$ , where  $u_\rho$  and  $v_\rho$  denote the source and destination nodes, respectively.  $a_\rho$  is the transaction amount and  $t_\rho$  is the initiation time.

For each transaction  $\rho$ , a set of candidate paths  $\mathcal{Q}^\rho$  connecting  $u_\rho$  to  $v_\rho$  is assumed. Each path  $q \in \mathcal{Q}^\rho$  is defined as  $q = (u_\rho, v_1^q, v_2^q, \dots, v_l^q, v_\rho)$ , where  $v_1^q, v_2^q, \dots, v_l^q$  are intermediate nodes along the path.

The proposed routing algorithm supports multipath payments. A single transaction is segmented into multiple sub-transactions routed through distinct paths. Let  $z_{\rho,q}$  denote the sub-amount of transaction  $\rho$  assigned to path  $q$ . A binary variable  $\delta_\rho \in \{0, 1\}$

is defined to indicate whether transaction  $\rho$  is successfully completed ( $\delta_\rho = 1$ ) or not ( $\delta_\rho = 0$ ).

The optimization problem is formulated with the following objectives:

1. **Maximize the total amount of successfully completed transactions.** As delineated in Equation 3.1, this objective seeks to prioritize the successful completion of transactions for increasing the total transferred amounts.

$$\max \sum_{\rho \in \mathcal{R}} \sum_{q \in \mathcal{Q}^\rho} z_{\rho,q} \cdot \delta_\rho \quad (3.1)$$

2. **Minimize the total transaction fees.** As presented in Equation 3.2, this objective aims to reduce the total cost incurred from routing payments.  $C(z_{\rho,q}, q)$  denotes the fee for sending amount  $z_{\rho,q}$  along path  $q$ .

$$\min \sum_{\rho \in \mathcal{R}} \sum_{q \in \mathcal{Q}^\rho} C(z_{\rho,q}, q) \quad (3.2)$$

## Constraints

The optimization is subject to the following constraints:

1. **Transaction fulfillment constraint.** As expressed in Equation 3.3, the total allocated sub-amounts for each transaction must meet or exceed its required amount.

$$\sum_{q \in \mathcal{Q}^\rho} z_{\rho,q} \geq a_\rho \cdot \delta_\rho, \quad \forall \rho \in \mathcal{R} \quad (3.3)$$

2. **Channel capacity constraint.** For each channel  $(i, j)$  with available capacity  $cap_{i,j}$ , the aggregate flow through the channel from all transactions must not exceed this capacity. The variable  $z_{\rho,q}$  is included in the sum if  $(i, j) \in q$ .

$$\sum_{\rho \in \mathcal{R}} \sum_{q \in \mathcal{Q}^\rho: (i,j) \in q} z_{\rho,q} \leq cap_{i,j}, \quad \forall (i, j) \in E \quad (3.4)$$

3. **Non-negativity constraint.** All allocated sub-transaction amounts must be non-negative.

$$z_{\rho,q} \geq 0, \quad \forall \rho \in \mathcal{R}, \quad q \in \mathcal{Q}^\rho \quad (3.5)$$

4. **Binary success indicator.** The transaction success variable  $\delta_\rho$  is constrained to be binary.

$$\delta_\rho \in \{0, 1\}, \quad \forall \rho \in \mathcal{R} \quad (3.6)$$

## Chapter 4

# Proposed Solution

This chapter provides a detailed explanation of the routing algorithm designed for PCN. The algorithm consists of three components: Routing Table Establishment, Route Analysis and Parallel Payment and Probing, and Failed Handling. The complete system architecture is illustrated in Figure 4.1.

### 4.1 Notation and Definitions

Table 4.1 lists the primary symbols used throughout the proposed routing algorithm. These notations are essential for the detailed operations in each phase.

### 4.2 Routing Table Establishment

This study assumes that each node within the Payment Channel Network (PCN) can acquire the overall network topology and initial channel capacities via the gossip protocol. In the initial phase of the system, a routing table is constructed for each node by executing the candidate path computation process defined in Algorithm 1.

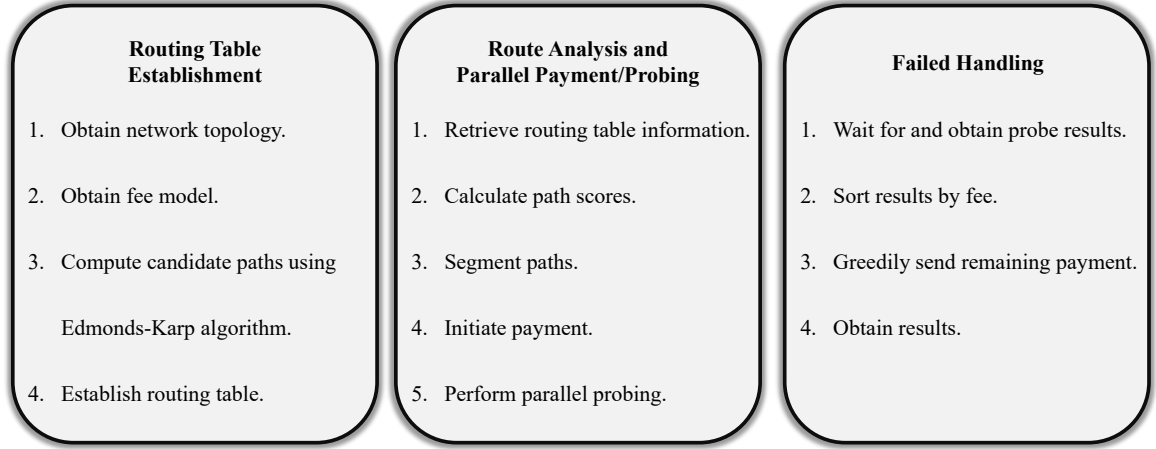


Figure 4.1: Flow chart of the system.

TABLE 4.1: List of Notations

| Symbol                | Description                                                                                         |
|-----------------------|-----------------------------------------------------------------------------------------------------|
| $p$                   | A specific path in the payment network                                                              |
| $p_i$                 | An individual path (often indexed, e.g., $p_1, p_2, \dots$ )                                        |
| $Y(p)$                | Capacity of path $p$                                                                                |
| $A$                   | Total transaction amount to be split                                                                |
| $A_i$                 | Amount of the transaction allocated to path $p_i$                                                   |
| $k$                   | Maximum number of paths to use for splitting, or number of top paths to probe                       |
| $\mathcal{P}$         | Set of all candidate paths for the transaction                                                      |
| $\mathcal{S}$         | Subset of currently selected paths from $\mathcal{P}$                                               |
| $\text{TotalFee}(p)$  | Total forwarding cost for path $p$                                                                  |
| $\alpha_{[v_i, v_j]}$ | Base fee for channel $(v_i, v_j)$                                                                   |
| $\beta_{[v_i, v_j]}$  | Fee rate for channel $(v_i, v_j)$                                                                   |
| $G$                   | The current state of the network graph                                                              |
| $A_{rem}$             | Remaining transaction amount that needs to be paid                                                  |
| $\mathcal{R}$         | Set of successful probing results, each containing a path $p_i$ and its confirmed capacity $Y(p_i)$ |
| $\gamma$              | Splitting threshold                                                                                 |

---

**Algorithm 1:** Candidate Path Computation from Node  $v_s$ 


---

**Input:**

Payment channel graph  $G = (V, E)$ ;

Channel capacities  $c_{[v_i, v_j]}$ , fees  $\alpha_{[v_i, v_j]}$ ,  $\beta_{[v_i, v_j]}$ ;

Source node  $v_s$ .

**Output:**

Edge-disjoint candidate paths  $\{\mathcal{P}_{v_s \rightarrow v_d} \mid v_d \in V, v_d \neq v_s\}$ .

**Definition:**

$\mathcal{P}_{v_s \rightarrow v_d}$ : Set of edge-disjoint candidate paths from  $v_s$  to  $v_d$ .

$G^{res}$ : Residual graph with updated capacities after each path discovery.

$p$ : A path from  $v_s$  to  $v_d$ , found via BFS.

TotalFee( $p$ ) =  $\sum_{(v_i, v_j) \in p} \alpha_{[v_i, v_j]} + \beta_{[v_i, v_j]} \cdot A$ : Forwarding cost of  $p$  for amount  $A$ .

```

1 foreach  $v_d \in V, v_d \neq v_s$  do
2   Initialize residual graph  $G^{res}$  with capacities  $c_{[v_i, v_j]}$ 
3    $\mathcal{P}_{v_s \rightarrow v_d} \leftarrow \emptyset$ 
4   while a path  $p$  exists from  $v_s$  to  $v_d$  in  $G^{res}$  via BFS do
5     Let  $p = \{v_s, \dots, v_d\}$ 
6      $Y(p) \leftarrow \min\{c_{[v_i, v_j]} \mid (v_i, v_j) \in p\}$ 
7     if  $Y(p) > 0$  then
8        $A \leftarrow Y(p)$ 
9        $\alpha(p) \leftarrow \sum_{(v_i, v_j) \in p} \alpha_{[v_i, v_j]}$ 
10       $\beta(p) \leftarrow \sum_{(v_i, v_j) \in p} \beta_{[v_i, v_j]}$ 
11      Add  $(p, A, \alpha(p), \beta(p))$  to  $\mathcal{P}_{v_s \rightarrow v_d}$ 
12      foreach  $(v_i, v_j) \in p$  do
13         $c_{[v_i, v_j]} \leftarrow c_{[v_i, v_j]} - A$ 
14         $c_{[v_j, v_i]} \leftarrow c_{[v_j, v_i]} + A$ 
15      else
16        Identify bottleneck edge  $(v_b, v'_b)$  on  $p$  with minimal capacity
17        Set  $c_{[v_b, v'_b]} \leftarrow 0$  in  $G^{res}$ 
18      Remove all edges  $(v_i, v_j)$  from  $G^{res}$  where  $c_{[v_i, v_j]} \leq 0$ 
19 return  $\{\mathcal{P}_{v_s \rightarrow v_d} \mid v_d \in V, v_d \neq v_s\}$ 

```

---

To discover multiple candidate paths to each destination node  $v_d$  (where  $v_d \neq v_s$ ), this study proposes a breadth-first search (BFS)-based edge-disjoint path computation algorithm. The algorithm first initializes a residual graph based on the original channel capacities  $c_{[v_i, v_j]}$ , and then repeatedly performs BFS to identify feasible paths from the source node  $v_s$  to  $v_d$ .

Once a path  $p$  is identified, the algorithm calculates the maximum transferable amount  $Y(p)$  along the path, which is defined as the minimum residual capacity among all edges on the path. If  $Y(p) > 0$ , the algorithm further computes the total base fee and total fee rate along the path, and then stores the path along with its attributes as a candidate path.

The residual graph is subsequently updated based on  $Y(p)$ . The capacity of each forward edge is reduced by  $Y(p)$ , while the capacity of the corresponding reverse edge is increased by  $Y(p)$  to support potential backward transmissions. If  $Y(p) = 0$ , the algorithm identifies the bottleneck edge and removes it from the residual graph to facilitate the discovery of alternative paths.

## 4.3 Phase I: Routing Analysis with Concurrent Payment and Probing

### 4.3.1 Path Scoring

The algorithm proposes a composite scoring function to prioritize candidate paths for a given transaction request. Each path  $p$  is evaluated based on three criteria: channel capacity, transaction fee, and channel usage frequency. The scoring function is defined as follows:

$$\text{Score}(p) = \alpha \cdot \widetilde{Y}(p) - \beta \cdot \widetilde{F}(p) - \gamma \cdot \widetilde{U}(p) \quad (4.1)$$

where:

- $\widetilde{Y}(p)$  is the normalized maximum payable amount along path  $p$  (i.e., normalized  $Y(p)$ );
- $\widetilde{F}(p)$  is the normalized total forwarding fee to send amount  $A$  along  $p$ , computed as  $\sum_{(v_i, v_j) \in p} (\alpha_{[v_i, v_j]} + \beta_{[v_i, v_j]} \cdot A)$ ;
- $\widetilde{U}(p)$  is the normalized maximum usage frequency among all channels on path  $p$ , defined as  $\max_{(v_i, v_j) \in p} U_{[v_i, v_j]}$ ;
- $\alpha, \beta, \gamma$  are user-defined weighting parameters that control the trade-off between capacity, fee, and congestion.

Each candidate path is assigned a score utilizing Equation 4.1. All paths are subsequently sorted in descending order of their scores. The top-ranked paths are preferred for subsequent payment and probing procedures.

#### 4.3.2 Channel Usage Frequency Update Mechanism

To quantify the usage frequency of each channel, as referenced in the path scoring model, each node within the system locally maintains historical records of its owned channels. Whenever a channel  $(u, v)$  is utilized for a transaction, the node updates its usage frequency based on the elapsed time since its last usage. Specifically, an *Exponentially Weighted Moving Average (EWMA)* is applied to emphasize recent activity and to adapt to temporal usage dynamics.

This mechanism enables the system to reflect the real-time activeness of each channel and to incorporate such information into routing decisions. The frequency update procedure is formally described in Algorithm 2.

---

**Algorithm 2:** Update Usage Frequency of Channel  $U_{[v_i, v_j]}$

---

**Input:**

Channel  $(v_i, v_j)$ ;

EWMA update parameter  $\lambda \in (0, 1)$

**Output:**

Updated  $U_{[v_i, v_j]}$  and last-used timestamp

**Definition:**

$U_{[v_i, v_j]}$ : Estimated usage frequency of channel  $(v_i, v_j)$  based on Exponentially Weighted Moving Average (EWMA).

$last\_used_{[v_i, v_j]}$ : The last time channel  $(v_i, v_j)$  was used.

$\lambda$ : Smoothing factor for EWMA, balancing recent usage and historical average.

```

1 now  $\leftarrow$  current system time
2 if channel  $(v_i, v_j)$  was previously used then
3    $\Delta t \leftarrow now - last\_used_{[v_i, v_j]}$ 
4   if  $\Delta t > 0$  then
5      $U_{[v_i, v_j]} \leftarrow \lambda \cdot \frac{1}{\Delta t} + (1 - \lambda) \cdot U_{[v_i, v_j]}$ 
6 else
7    $U_{[v_i, v_j]} \leftarrow 1$  // Initialize on first use
8  $last\_used_{[v_i, v_j]} \leftarrow now$ 

```

---

#### 4.3.3 Trusted Nodes for Real-Time Channel State Inquiry

To enhance the timeliness and accuracy of channel state information, the proposed system introduces the concept of *trusted nodes*. A trusted node refers to a counterparty that has exhibited a frequent transaction history with the sender. In practice, the trusted nodes may be individuals or merchants with whom the user has established trust, such as friends, family, or business partners.

The system offers an API interface that enables the sender to query trusted nodes in real-time for up-to-date information regarding the residual capacity and usage frequency of specific payment channels. This mechanism complements the static topology disseminated through the gossip protocol. Routing decisions can be made by the more updated information.

---

**Algorithm 3:** Adaptive Multi-Path Payment Splitting

---

**Input:**Candidate paths  $\mathcal{P} = \{p_1, p_2, \dots\}$  with capacities  $Y(p)$ ;Transaction amount  $A$ ;Minimum success rate threshold  $\theta$ ;Maximum path count  $k$ **Output:**Split amounts  $\{A_i \mid p_i \in \mathcal{S}\}$ **Definition:** $\theta$ : Minimum success rate threshold for path selection $\mathcal{S}$ : Subset of selected paths from  $\mathcal{P}$  $T$ : Total capacity of the currently selected paths in  $\mathcal{S}$  $\rho$ : Minimum remaining capacity ratio among selected paths

```
1 Initialize  $\mathcal{S} \leftarrow \{p_1\}$ 
2 while True do
3    $T \leftarrow \sum_{p_i \in \mathcal{S}} Y(p_i)$ 
4   if  $T = 0$  then
5     raise error: insufficient capacity
6   foreach  $p_i \in \mathcal{S}$  do
7      $A_i \leftarrow A \cdot \frac{Y(p_i)}{T}$ 
8    $\rho \leftarrow \min_{p_i \in \mathcal{S}} \left( \frac{Y(p_i) - A_i}{Y(p_i)} \right)$ 
9   if  $\rho \geq \theta$  or  $|\mathcal{S}| = |\mathcal{P}|$  or  $|\mathcal{S}| = k$  then
10    break
11   Add next path  $p_j$  to  $\mathcal{S}$ 
12 return  $\{A_i \mid p_i \in \mathcal{S}\}$ 
```

---

By leveraging trusted nodes, the sender can enhance the quality of path selection and traffic distribution. Payment success rates and overall routing efficiency can be improved.

#### 4.3.4 Adaptive Multi-Path Payment Splitting Mechanism

An adaptive multi-path splitting mechanism is designed to efficiently allocate payment amounts across multiple paths, as presented in Algorithm 3. The algorithm dynamically assigns portions of the total payment to candidate paths based on their maximum transferable capacities. The algorithm also ensures that the minimum success rate among the selected paths does not fall below the system-defined threshold  $\theta$ .

### 4.3.5 Path Probing and Payment Execution Procedure

Algorithm 4 outlines the procedure for executing payment transfers and initiating path probing. The system first attempts to transmit the segmented payment amounts through the initially selected paths.

Subsequently, the system evaluates the remaining unutilized candidate paths, ranks them based on their total forwarding fees that comprise both fixed and proportional components. The top- $k$  paths with the lowest forwarding fees are selected and issued as probing tasks to collect up-to-date information on channel availability.

Finally, if the payment is successfully completed via the initially selected paths, the task will be designated as successful and the total incurred fees will be calculated. Conversely, if the available capacity is insufficient, the task will be designated as failed and the subsequent retry mechanism for failed payments will be proceeded.

## 4.4 Phase II: Failed Handling

Algorithm 5 introduces the fallback mechanism that is invoked after the failure of the initial routing attempt. The system utilizes previously collected probing results to identify alternative paths with sufficient capacity. For each probed path, a threshold is applied to the probed capacity to reduce the likelihood of failure during reattempts. Based on this threshold-adjusted capacity, the system allocates the remaining unpaid amount across eligible fallback paths.

Each selected fallback path subsequently executes the payment. If the entire remaining amount is successfully transferred via these paths, the payment task will be designated as success. If only a portion of the amount is transferred, the entire transaction will be defined as failure.

---

**Algorithm 4:** Path Probing and Payment Execution

---

**Input:**Candidate paths  $\{(p, Y(p), \alpha(p), \beta(p))\}$  sorted by preference;Assigned split amounts  $\{A_i\}$  for selected paths**Output:**

Payment status (success/failure), total fee

**Definition:** $\alpha(p)$ : Base fee for path  $p$  (sum of base fees of channels in  $p$ ) $\beta(p)$ : Fee rate for path  $p$  (sum of fee rates of channels in  $p$ ) $\mathcal{C}$ : Set of remaining unselected candidate paths $\tau$ : Timeout parameter for probing taskProbeTask( $p, \tau$ ): A task to probe path  $p$  with timeout  $\tau$ 

// Step 1: Execute payment on selected paths

1 **foreach**  $(p_i, A_i)$  *in selected paths* **do**2     └ Attempt to transfer  $A_i$  through  $p_i$ 

// Step 2: Probe additional candidate paths by fee

3 Let  $\mathcal{C} \leftarrow$  remaining unselected candidate paths4 Sort  $\mathcal{C}$  by total forwarding cost:

$$// \text{TotalFee}(p) = \sum_{(v_i, v_j) \in p} \alpha_{[v_i, v_j]} + \beta_{[v_i, v_j]} \cdot A$$

5 Let  $\mathcal{C}_{top} \leftarrow$  top  $k$  paths with lowest total fee in  $\mathcal{C}$ 6 **foreach**  $p \in \mathcal{C}_{top}$  **do**7     └ Create and enqueue probing task ProbeTask( $p, \tau$ )

// Step 3: Finalize payment result

8 **if** *channel capacity is sufficient* **then**

9     └ Mark task as successful; compute total fee

10 **else**

11     └ Mark task as failed due to insufficient capacity

12 **return** *Payment status and total fee*

---

---

**Algorithm 5:** Fallback Routing Using Probed Paths

---

**Input:**

Remaining amount  $A_{rem}$ ;  
Probing results  $\mathcal{R} = \{(p_i, Y(p_i))\}$ ;  
Splitting threshold  $\gamma$ ;  
Network graph  $G$  and current payment task

**Output:**

Updated payment task (success/failure), total fee

**Definition:**

Current payment task: The overall payment operation being processed

$\mathcal{F}$ : List of fallback paths chosen for payment, each with an assigned amount  $(p, A_s)$

$A_c$ : Current remaining capacity/amount to fulfill from  $A_{rem}$

$A_s$ : Amount to be sent through a specific fallback path  $p$

success: Boolean variable indicating the overall success of fallback routing

$fee_{total}$ : Accumulative total fee incurred for successful transfers

```
1 Initialize  $\mathcal{F} \leftarrow []$ 
2  $A_c \leftarrow A_{rem}$  // Remaining amount to fulfill
3 foreach  $(p, Y(p)) \in \mathcal{R}$  do
4   if  $A_c \leq 0$  then
5     break // Target amount fulfilled
6    $A_s \leftarrow \min(A_c, Y(p) \cdot (1 - \gamma))$  // Apply success margin
7   if  $A_s > 0$  then
8     Append  $(p, A_s)$  to  $\mathcal{F}$ 
9      $A_c \leftarrow A_c - A_s$ 
10 if  $\mathcal{F}$  is empty then
11   Mark payment task as failed; no feasible fallback paths
12 else
13   success  $\leftarrow$  True,  $fee_{total} \leftarrow 0$ 
14   foreach  $(p, A_s) \in \mathcal{F}$  do
15     if transfer of  $A_s$  through  $p$  is successful then
16       Update network state;  $fee_{total} += \text{TotalFee}(p)$ 
17     else
18       success  $\leftarrow$  False; break // Transfer failed, stop further attempts
19   if success and  $A_c \leq 0$  then
20     Mark payment task as successful; return  $fee \leftarrow fee_{total}$ 
21   else if success and  $A_c > 0$  then
22     Mark payment as partially successful; update  $A_{rem} \leftarrow A_c$ 
23   else
24     Mark payment task as failed; preserve  $A_{rem}$ 
```

---

## Chapter 5

# Performance Evaluation

The evaluation of the system was performed on Python 3.12.3 running on an AMD Ryzen 5 3600 CPU. A network comprising 100 nodes was generated using the Barabasi-Albert model [?], with the average node degree configured to 5. The channel capacity for each channel was set using the data from bitcoinvisuals.com [?]. Channel transaction fees were generated based on the average fee values and median values provided by 1ml.com [?]. For transaction amounts, this study sampled credit card transaction amounts from the Credit Card Fraud Detection dataset [?]. Transaction partner combinations were generated with the research data from Flash [?]. This approach aimed to simulate realistic transaction patterns. Regarding the simulator setup, this system employed Poisson distribution simulations to randomly generate transaction arrival times for various transaction frequencies (TPS). The single-hop latency in the experiment was set to 30 ms, referencing Deter-Pay [?]. Additionally, transactions were enabled to execute concurrently.

TABLE 5.1: Simulation Parameters

| Parameter                   | Value              |
|-----------------------------|--------------------|
| Number of Nodes             | 100                |
| Number of Channels          | 521                |
| Average Channel Capacity    | 8,929,583 satoshis |
| Median Channel Capacity     | 1,589,612 satoshis |
| Transaction Frequency (TPS) | 10—100             |
| Transaction Amount Range    | 0.0—25,691.16 USD  |
| Simulation Duration         | 30—60 s            |
| Single-hop Latency          | 30 ms              |

## 5.1 Comparison Algorithms

1. **Shortest Path:** Shortest path routing is currently employed in the Lightning Network to determine the path between senders and receivers for payments.
2. **Flash** [?]: Flash differentiates between high-value ???elephant??? payments and low-value ???mice??? payments. The strategy can improve success rates.
3. **Deter-Pay** [?]: Deter-Pay combines precomputed candidate paths with a probing-based balance reservation mechanism.

## 5.2 Performance Under Different Workloads

The experiment aims to verify the algorithm’s comprehensive performance under varying transaction frequencies. The experimental setup configures the transaction frequency per second from 10 TPS to 100 TPS. The transaction success rate, execution time, and transaction fees incurred per satoshi are recorded with the varying frequencies.

**Transaction Success Rate:** Figure 5.1 illustrates the variation in transaction success rate as the workload (TPS) increases. The proposed method consistently outperforms other protocols, including Deter-Pay, Flash, and Shortest Path, with all tested work-

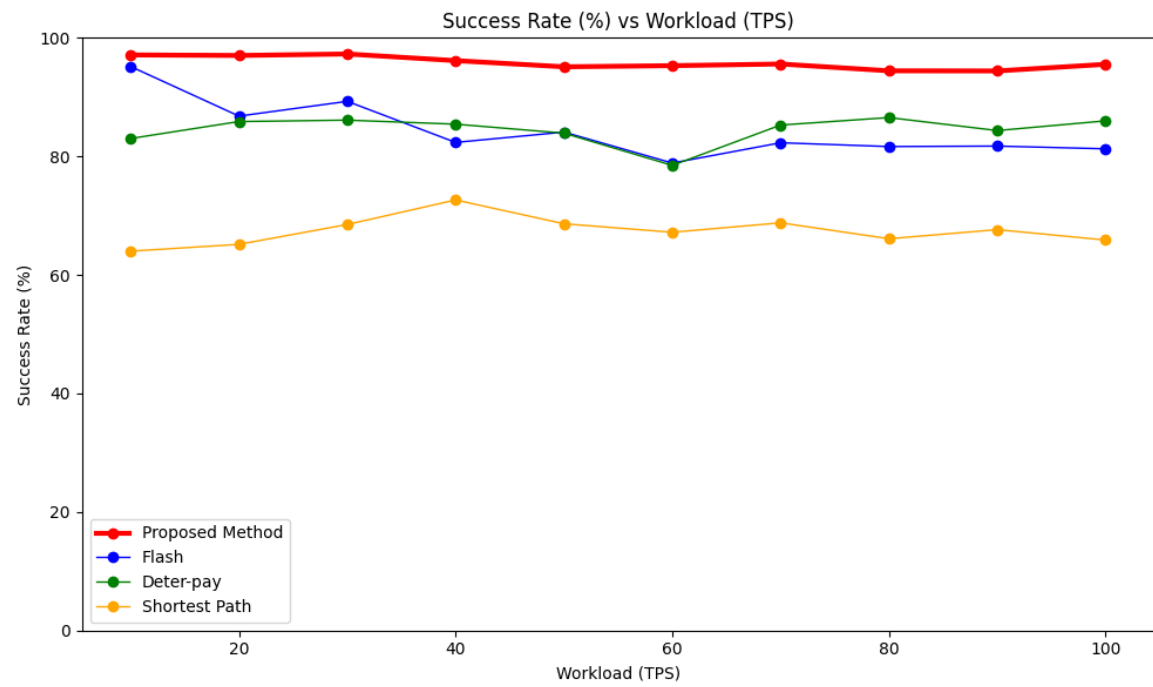


Figure 5.1: Success rate under different workloads.



Figure 5.2: Execution time under different workloads.

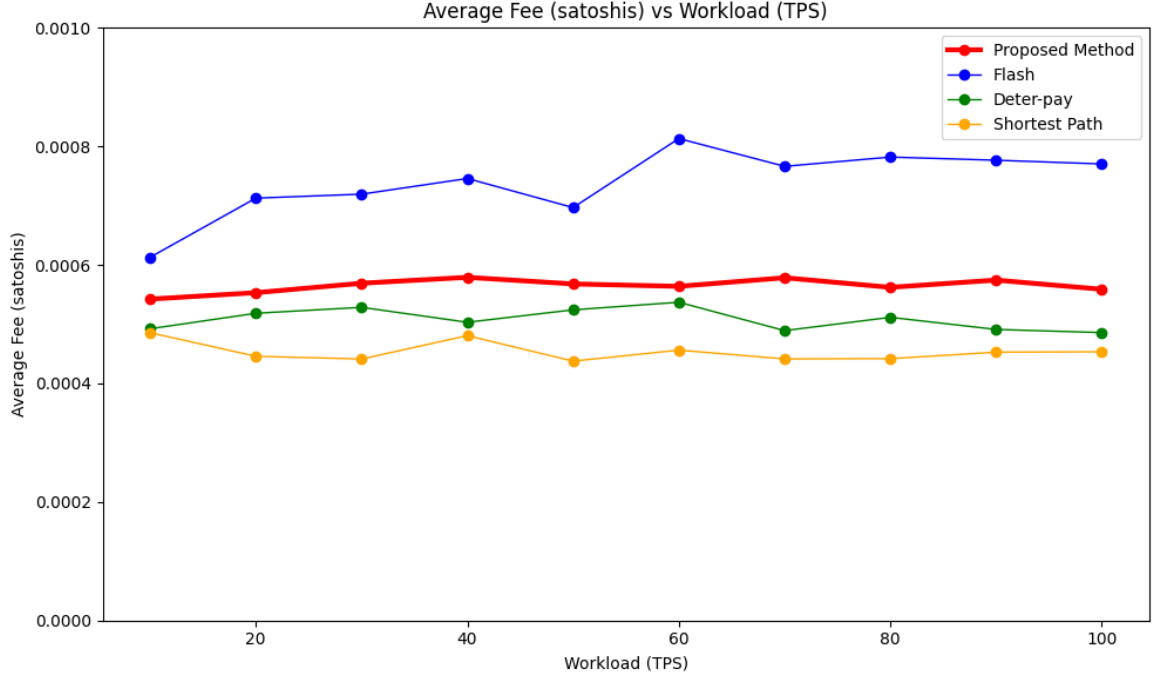


Figure 5.3: Transaction fees under different workloads.

loads. The method achieves the highest success rate even at a transaction frequency of 100 TPS and demonstrates its efficiency under high-load conditions. In contrast, Deter-Pay and Flash exhibit fluctuating or declining success rates as the workload increases, and Shortest Path consistently has the lowest performance among all evaluated methods.

Transaction Latency: Figure 5.2 displays the variation in transaction delay with different transaction workload frequencies. Compared to Deter-Pay and Flash, the proposed method exhibits stable and comparatively lower execution times. Shortest Path that depends solely on the most basic shortest path algorithm based on network topology, achieves the highest execution efficiency. The proposed method employs a two-phase payment process. A successful first payment phase can reduce overall transaction delay. Deter-Pay’s requirement for path probing and channel reservation contributes to its higher transaction delay, especially as higher transaction frequen-

cies may necessitate multiple probing and reservation attempts. Flash exhibits lower latency than Deter-Pay, but the method may incur higher delays due to repeated attempts. Flash lacks effective adaptation because of prior failures.

Transaction Fees: Figure 5.3 illustrates the variation in transaction fees under different transaction workloads. The proposed method incurs slightly higher fees than Deter-Pay but demonstrates a significant fee advantage over Flash. Shortest Path consistently yields the lowest transaction fees. The result is attributed to its reliance on a single path and a short hop count.

### 5.3 Performance with Scaled Capacities

Currently, the average channel capacity of the Lightning Network is approximately 900,000 Satoshis. As off-chain payments become more widespread and more PCNs are practically deployed, channel capacities are expected to increase. This experiment was conducted to verify the effectiveness of the proposed method in the future development of payment channels. The simulation was examined with channel capacities scaled from one-fold to tenfold. These simulations were performed with a fixed testing duration and a constant transaction frequency.

Figure 5.4 illustrates that the transaction success rates of all routing methods increase with the enhancement of channel capacity. Under conditions of increased channel capacity, the proposed method demonstrates a significant lead over Deter-Pay and Shortest Path methods.

Figure 5.5 indicates a gradual reduction in the execution time of the proposed method as channel capacity increases. This reduction is attributed to the increased likelihood of successful transactions in the algorithm's first payment phase with the

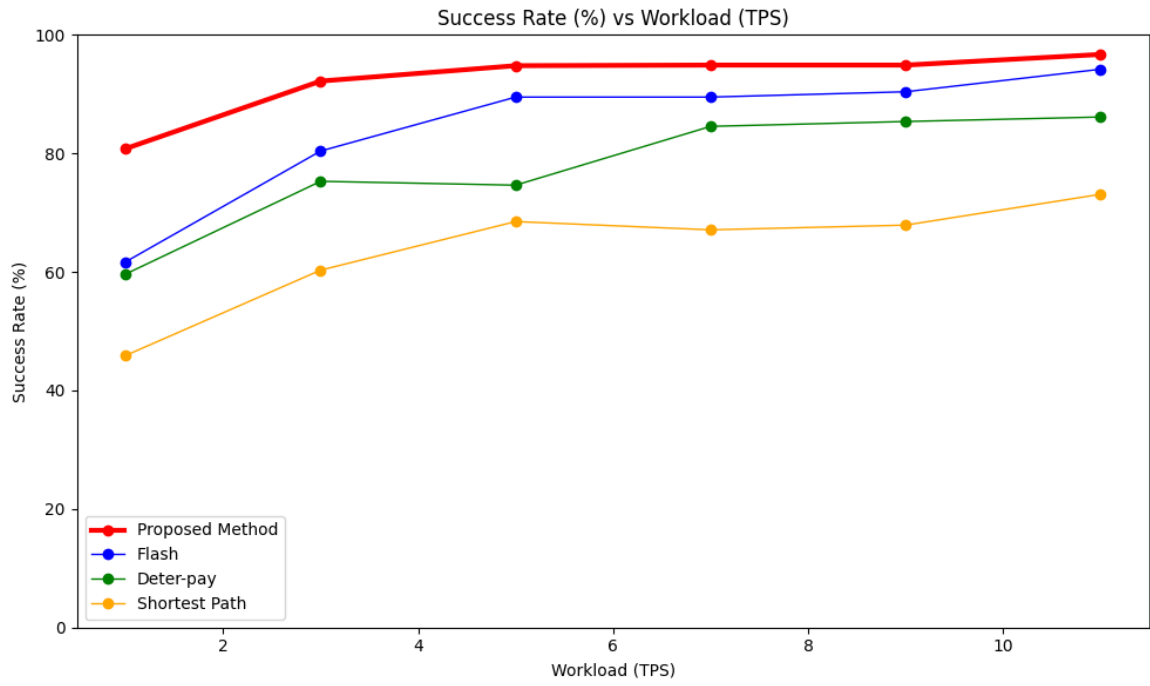


Figure 5.4: Success rate under different channel sizes.

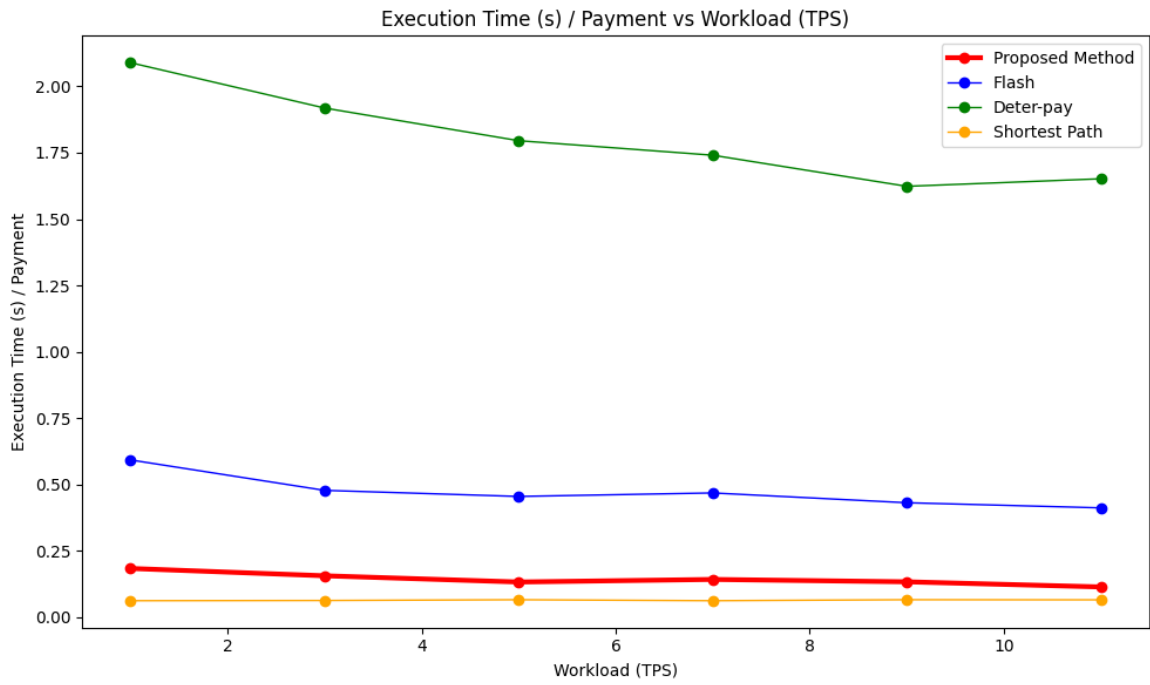


Figure 5.5: Execution time under different channel sizes.

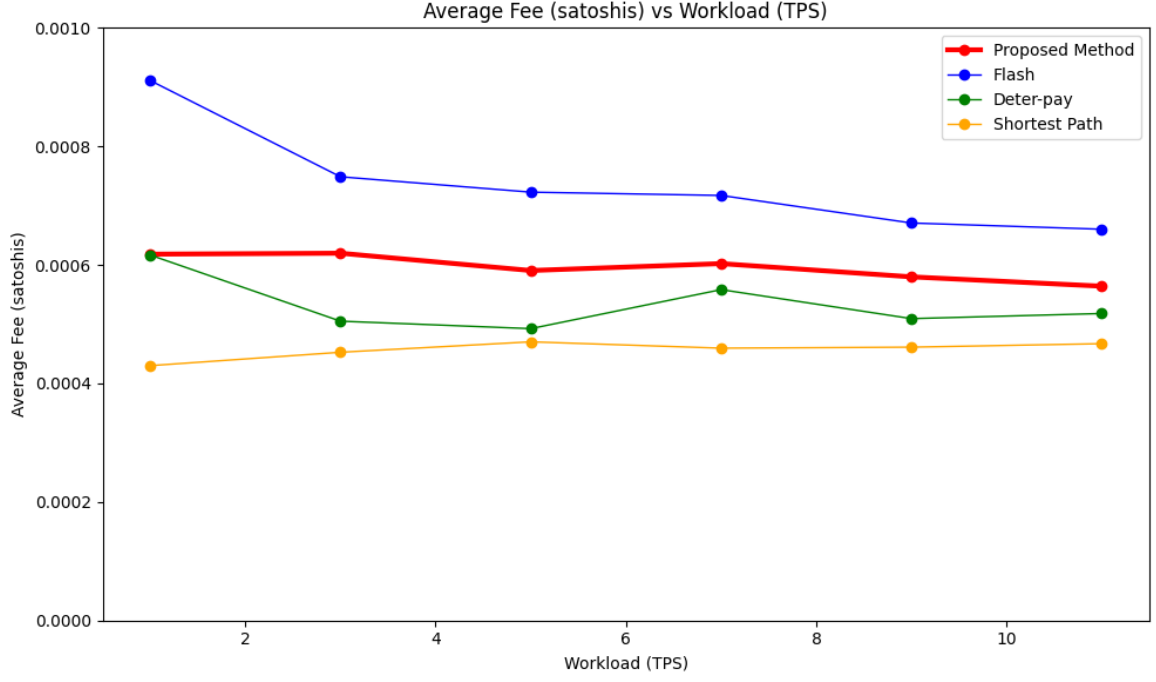


Figure 5.6: Transaction fees under different channel sizes.

larger channel capacity. Path score calculation and transaction splitting are used to achieve this success. This approach reduces the need for second-stage failed handling and consequently lowers transaction delay. In terms of transaction delay, the algorithm exhibits a more significant advantage compared to Deter-Pay and Flash.

Finally, regarding transaction fees, Figure 5.6 shows that the proposed method offers a lower fee advantage compared to Flash. Although its fees are higher than those of Deter-Pay and Shortest Path, the method demonstrates a significant advantage in success rate. The algorithm effectively balances success rate, execution time, and transaction fees.

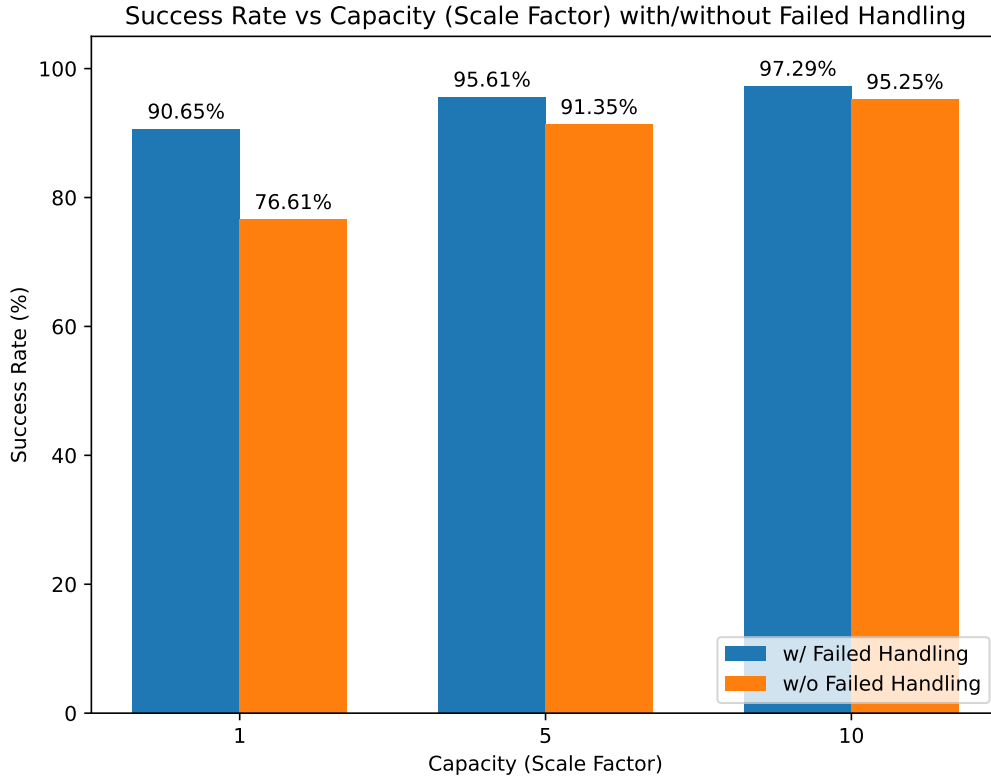


Figure 5.7: Success rate with and without failed handling mechanisms.

## 5.4 Performance with Failed Handling Mechanisms

The experiment was designed to verify the effectiveness of the algorithm’s second phase, called Phase II: Failed Handling. The experiments with or without Phase II were conducted. The results were subsequently tested under different Capacity Scales.

Figure 5.7 reveals that under smaller channel capacity scale factors, the Phase II: Failed Handling mechanism significantly enhances the transaction success rate. When channel capacity is larger, the improvement in success rate by failed handling becomes less prominent. Most transactions can be completed in the first phase. This experimental result corroborates the findings in Section 5.3 regarding lower transaction delay with larger channel capacity. This outcome is similarly attributed to the increased success rate in the first phase.

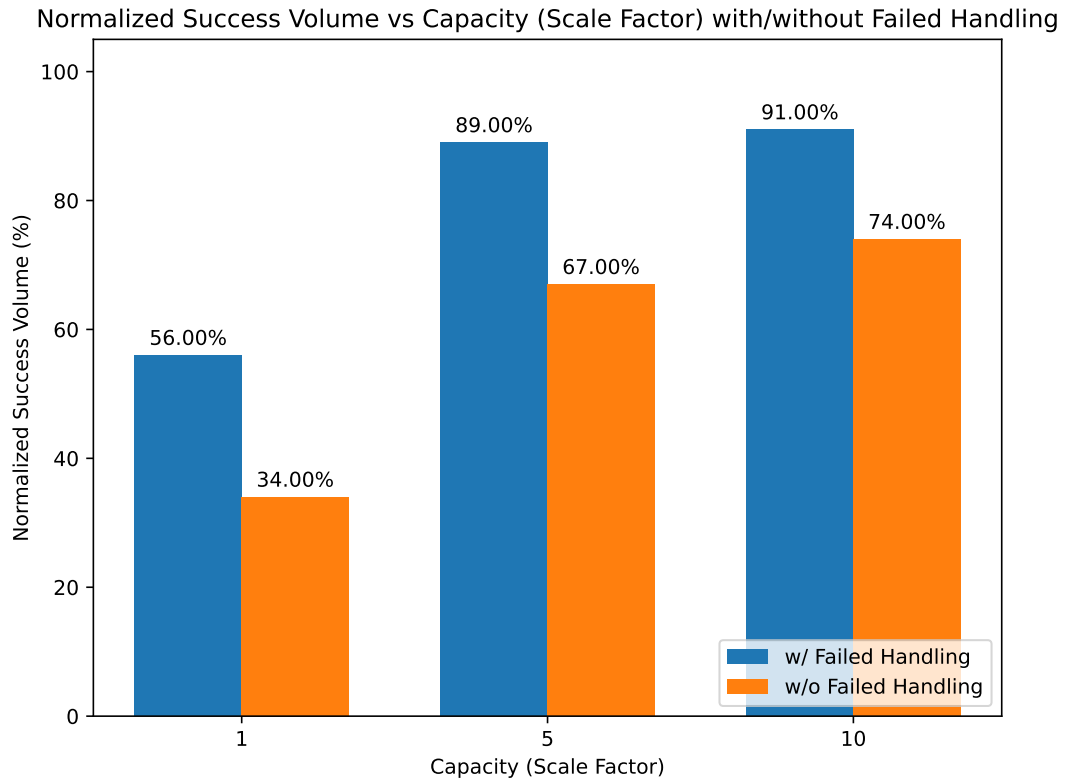


Figure 5.8: Successful volume with and without failed handling mechanisms.

Figure 5.8 demonstrates a significant increase in the total successful transaction volume ratio regardless of the channel capacity. The Phase II failed handling method effectively satisfies large transaction demands and enhances transaction completion rates. The results are achieved by enabling re-attempts based on probing results and intelligently allocating large transaction amounts to available channels.

## Chapter 6

# Conclusion and Future Work

### 6.1 Conclusion

This thesis presents a two-phase routing algorithm to improve efficiency and reliability in PCNs. The algorithm consists of an initial payment attempt and a re-exploration phase triggered upon failure. Adaptive responses can be enabled to dynamic or inconsistent network conditions.

Routing decisions are informed by a combination of factors, including channel capacity, balance status, fee conditions, and usage patterns. The integrated approach improves path selection, reduces payment failure rates, and lowers overall transaction costs.

Routing performance was examined based on the simulated transaction workloads and network conditions. The proposed algorithm typically outperforms existing methods in terms of success rate, latency, and cost. The evaluation results confirm the effectiveness of the algorithm for scalable PCN routing.

## 6.2 Future Work

In this research, it is assumed that channel-related information can be obtained through trusted nodes. Nevertheless, considering the high demands for privacy protection in decentralized platforms, it is imperative that no sensitive data is compromised through any means. Therefore, future work ought to explore the development of more efficient routing mechanisms and also guarantee the privacy of every node.

# References

- [1] Z. Qin, A. Li, C. Dong, H. Dai, and Z. Xu, “Completion time minimization for Multi-UAV information collection via trajectory planning,” *Sensors*, vol. 19, no. 18, p. Art. no. 4032, Sept. 2019.
- [2] L. Zhang, C. He, Y. Peng, Z. Liu, and X. Zhu, “Multi-UAV data collection and path planning method for large-scale terminal access,” *Sensors*, vol. 23, no. 20, p. Art. no. 8601, Oct. 2023.
- [3] K. Xiang and Y. He, “Uav-assisted MEC system considering UAV trajectory and task offloading strategy,” in *Proceedings of 2023 IEEE International Conference on Communications (ICC): Green Communication Systems and Networks Symposium*, May 2023, pp. 4677–4682.